Hotel API Deployment Guide: Ubuntu VirtualBox and MonsterASP.NET

Step-by-step deployment, configuration, and troubleshooting for the Hotel API

This guide combines the Ubuntu/Docker deployment chat and the MonsterASP.NET troubleshooting chat. The project is a .NET 10 API that uses SQL Server, JWT authentication, and SMTP email.

Route	What runs where	Best fit
Ubuntu in VirtualBox	Docker runs the API and SQL Server inside your Ubuntu VM	A local network demo or learning deployment that you control
MonsterASP.NET	The API runs as an ASP.NET Core website under IIS; the database is hosted separately by MonsterASP	A hosted website managed through the control panel

MonsterASP does not support Docker containers. Publish the API as a normal ASP.NET Core website there; use Docker only for the Ubuntu route. MonsterASP supported technologies

Before either route: protect the project credentials

The project currently has database and mail settings in tracked configuration files. Treat any password that has been committed as exposed: replace it with a new one before using the project publicly, and keep the new value out of Git and this guide. The local `Auth__CodeHmacKey` previously shown in chat should also be replaced with a new private key.

For MonsterASP, store the replacement settings in that website's **Environment variables** page. For automatic Ubuntu deployment, store the HMAC key as a GitHub Actions repository secret and pass it to the container as shown below. Never paste a real secret into a workflow log, source file, or chat.

Route 1: Ubuntu Server in VirtualBox with Docker

This route uses the existing Compose project and is suitable for a LAN demo. It does not configure a public domain or HTTPS. Do not expose the development setup directly to the public internet.

1. Create or check the Ubuntu VM

1. Use an Ubuntu Server LTS VM in VirtualBox. The earlier guide used 4 GB RAM, 2 CPUs, and a 40 GB virtual disk.
2. During Ubuntu setup, create the one Linux account you will use and select **Install OpenSSH server**. No extra Ubuntu user is needed.
3. Shut down the VM. In **VirtualBox > Settings > Network > Adapter 1**, select **Bridged Adapter**, choose the computer's active Wi-Fi or Ethernet adapter, and check **Cable Connected**.
4. Start Ubuntu and find its current LAN address:

```
ip -4 -br addr
```

Use the LAN address on the bridged network interface, usually in a range like `192.168.x.x`. Do not use `127.0.0.1`.

If there is no LAN address, check the interface and the Netplan filename first:

```
ip -br link
ls /etc/netplan
```

Open the file that actually exists. If it is `00-installer-config.yaml`, the command is:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Keep the existing settings and make sure the correct interface has `dhcp4: true`. For example, if `ip -br link` shows `enp0s3`:

```
network:
  version: 2
```

```
  ethernets:
    enp0s3:
      dhcp4: true
```

Replace `enp0s3` with the actual interface name. Save with **Ctrl+O**, Enter, then **Ctrl+X**. Apply and recheck:

```
sudo netplan try
ip -4 -br addr
```

2. Connect from Termius

If OpenSSH was not installed during Ubuntu setup, run this at the VM console:

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
```

In Termius, add a host with the VM's current LAN address, port `22`, the Ubuntu account created during setup, and that account's password. If Termius stops connecting after a reboot, check the address again in VirtualBox with `ip -4 -br addr` and update the saved Termius host; the bridged network may have assigned a different address.

3. Install Docker and get the project onto Ubuntu

In Termius, install Git and Docker:

```
sudo apt update
sudo apt install -y git
curl -fsSL https://get.docker.com | sudo sh
sudo docker run hello-world
```

The Docker convenience installer is intended for development and testing, which fits this local VM demo. Docker's Ubuntu installation guide

If the GitHub repository is public, clone it:

```
git clone https://github.com/gioninika/HotelFinalProject.git
cd HotelFinalProject
```

If the repository is private and you want to avoid configuring Git credentials on Ubuntu, use Termius' SFTP panel to copy the project folder from Windows, then `cd` to the copied folder. Run Compose from the folder containing `docker-compose.yml`.

4. Start the API and database

The API's `AuthService` requires `Auth:CodeHmacKey` to be at least 32 characters long. Without it, controller requests that construct the auth service can return 500, even though database migration commands succeeded. Pass the key to the container using a temporary Compose override from standard input. This does not create a `.env` file or add a Compose file to the project.

In the Ubuntu project folder, run the following. Enter the new key when prompted; `read -s` keeps it out of the terminal display and shell history.

```
read -rsp 'Paste a new private Auth__CodeHmacKey: ' Auth__CodeHmacKey
printf '\n'
export Auth__CodeHmacKey
sudo --preserve-env=Auth__CodeHmacKey docker compose \
  -f docker-compose.yml \
  -f docker-compose.override.yml \
  -f - up -d --build --remove-orphans <<'YAML'
services:
  hotel:
    environment:
      Auth__CodeHmacKey: ${Auth__CodeHmacKey:?Set the key in this shell}
YAML
unset Auth__CodeHmacKey
```

The existing `docker-compose.override.yml` is named explicitly because passing `-f` flags means Compose will not discover that override automatically. The last `-f -` reads the short, temporary override from standard input. Docker Compose CLI reference

Check that the services are running:

```
sudo docker compose ps
sudo docker compose logs --tail=100 hotel
```

From Windows, open:

```
http://<VM_LAN_IP>:8080/swagger
```

Replace `<VM_LAN_IP>` with the VM's current bridged address. SQL Server's port is not published by the current Compose setup; keep it that way for this LAN demo.

5. Update the containers manually

After copying updated files with SFTP, or after committing changes to `master` in a Git clone, go to the project folder and repeat the Compose command from step 4. The HMAC key must be passed again each time Compose recreates the API container. Then check `sudo docker compose ps` and test `/swagger` again.

The database uses a named Docker volume. Do not run `docker compose down -v`; `-v` removes the volume and its SQL data.

6. Optional: deploy from GitHub Actions

The project's current `.github/workflows/deploy.yml` runs on a push to `master` or a manual **Run workflow**. It uses a self-hosted Linux runner with the labels `self-hosted`, `linux`, `x64`, and `hotel-vm`.

1. In **HotelFinalProject > Settings > Actions > Runners**, add a Linux x64 self-hosted runner for this repository. Follow GitHub's setup page, use its temporary registration token, and assign the custom runner label `hotel-vm`.
2. In Ubuntu, from `~/actions-runner`, install and start the runner service:

   ```
   sudo ./svc.sh install
   sudo ./svc.sh start
   sudo ./svc.sh status
   ```

On an existing runner, use `sudo ./svc.sh status` and `sudo ./svc.sh start`. The service starts with Ubuntu. Do not also start `./run.sh` in another terminal; two processes for one runner registration cause the "A session for this runner already exists" error.

3. In **HotelFinalProject > Settings > Secrets and variables > Actions**, create a repository secret named `AUTH_CODE_HMAC_KEY`. Generate a new value privately; do not reuse the key shown in the old chat.
4. In `.github/workflows/deploy.yml`, give the build step access to that secret and pass it into the existing Compose stack. Replace the current **Build and start the app** step with:

```
      - name: Build and start the app
        env:
          Auth__CodeHmacKey: ${{ secrets.AUTH_CODE_HMAC_KEY }}
        run: |
          docker compose -f docker-compose.yml -f docker-compose.override.yml -f - up -d --build --remove-
orphans <<'YAML'
          services:
            hotel:
              environment:
                Auth__CodeHmacKey: ${Auth__CodeHmacKey:?Set AUTH_CODE_HMAC_KEY in repository secrets}
              restart: unless-stopped
            sqlserver:
              restart: unless-stopped
          YAML
          docker compose ps
```

This passes the key to Compose without saving a new Compose file. The restart policy also makes the API and database start after the VM restarts.

5. Keep the VM powered on and connected while a deployment is running. Merge `Test` into `master` with a pull request (**base:** `master`**, compare:** `Test`) to deploy those changes. A push to `Test` alone does not trigger this workflow.
6. Open **Actions > Deploy Hotel API** and read the run logs. When it succeeds, check **Settings > Actions > Runners** for **Online** and test `http://<VM_LAN_IP>:8080/swagger`.

"Listening for jobs" means the runner is idle and waiting. If a run is queued, check that this runner is registered under the same repository as the workflow and has the `hotel-vm` label. A repository-level runner for `HotelFinalProject` cannot take jobs from a different repository. A label disappearing is usually a sign that the runner registration was replaced; add `hotel-vm` to the active runner entry.

If the VM is off, start it in VirtualBox first; Termius cannot start it. In the Ubuntu console, check or start the runner with `cd ~/actions-runner && sudo ./svc.sh status && sudo ./svc.sh start`. Once GitHub shows the runner Online,

rerun a queued job or choose **Actions > Deploy Hotel API > Run workflow**. If you choose not to add the restart policy shown above, run the workflow again after every VM reboot to start the Compose services.

Route 2: publish the API to MonsterASP.NET

MonsterASP hosts regular ASP.NET Core apps under IIS and provides hosted SQL Server databases. It does not run your Docker Compose stack. MonsterASP publishing guide · Supported technologies

1. Prepare the MonsterASP website and database

1. In the MonsterASP control panel, create or open your website and its MSSQL database.
2. On the website's **Scripting** page, select the modern ASP.NET Core runtime matching the project's .NET 10 target. MonsterASP currently supports modern .NET versions.
3. In the database page, copy the connection string shown by MonsterASP. Do not reuse the old connection string committed in `appsettings.json`.

The current Free plan lists one website, 5 GB website storage, 256 MB dedicated RAM, one 1 GB database, limited traffic, and no MonsterASP email service. Check the current Free and Premium plan comparison before relying on it for a busy or always-on public API. Free websites and databases that receive no requests or connections for six months are archived. Free-service archive policy

2. Add production settings in the control panel

Open **Websites > your website > Scripting > Environment variables**. Add these values there; MonsterASP applies changes after you restart the application. ASP.NET Core maps the double underscore in names such as `Auth__CodeHmacKey` to a colon in configuration, such as `Auth:CodeHmacKey`. MonsterASP environment variables

Name	Value
`ASPNETCORE_ENVIRONMENT`	`Production`
`ConnectionStrings__DefaultConnection`	The connection string from this MonsterASP database, with the rotated password
`Auth__CodeHmacKey`	A new random private value at least 32 characters long
`Jwt__Key`	A new random signing key, at least 32 characters long
`Jwt__Issuer`	The issuer configured for this API's JWTs
`Jwt__Audience`	The audience configured for this API's JWTs

Generate the HMAC and JWT keys privately. For example, in PowerShell this creates a random 48-byte Base64 value and copies it to the clipboard without printing it:

```
$rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
$bytes = [byte[]]::new(48)
$rng.GetBytes($bytes)
[Convert]::ToBase64String($bytes) | Set-Clipboard
$rng.Dispose()
```

Paste a fresh value directly into the control panel, then clear the clipboard. Do not use the HMAC key shown in the earlier chat. Keep the JWT issuer and audience consistent with the application's expected token settings.

The API can apply migrations on startup only when `Database__ApplyMigrations` is enabled. If you already ran `dotnet ef database update` successfully against this same MonsterASP database, leave automatic migrations disabled unless you deliberately want the app to run them at startup.

3. Configure free email through Gmail

You do not need MonsterASP's paid email service to send the API's verification and password-reset messages. MonsterASP documents external SMTP for the Free plan, including Gmail with an App Password. MonsterASP Gmail SMTP instructions · SMTP plan options

1. Turn on 2-Step Verification in your Google account.

2. Create a Google App Password for the application. Do not use your normal Google password.
3. Add these environment variables in the same MonsterASP page:

Name	Value
`Email__Host`	`smtp.gmail.com`
`Email__Port`	`587`
`Email__UserName`	Your complete Gmail address
`Email__Password`	The Google App Password
`Email__FromAddress`	The same Gmail address
`Email__FromName`	`Hotel`
`Email__UseAuthentication`	`true`
`Email__UseStartTls`	`true`

4. Save and restart the website. Register a test guest, then check the inbox and spam folder for the verification message.

If Google does not offer App Passwords for your account, use another SMTP provider that permits authenticated SMTP. Do not substitute your normal Gmail password. The built-in MonsterASP SMTP service is a Premium option; Gmail SMTP uses your Gmail account instead.

4. Publish from Visual Studio

1. In the MonsterASP control panel, open **Websites > your website > Deploy > Visual Studio**. If you need a fresh profile, enable Web Deploy and download the `.publishSettings` file.
2. In Visual Studio, right-click the `Hotel` project, choose **Publish > Import profile**, and select that file. A publish profile contains deployment access details; keep it private and do not commit or share it.
3. Select the imported profile and publish the project in **Release** configuration. Visual Studio uploads the ASP.NET publish output to the website's `wwwroot` folder.
4. In MonsterASP, save the environment variables and click **Restart application**.
5. Open the website URL shown in the control panel and append `/swagger`. Test `GET /api/hotels`, then registration and email verification.

If a database migration succeeded but the endpoints still return 500, the migration only confirms that the database update ran; it does not prove that the application's controller services have their runtime secrets. The missing `Auth__CodeHmacKey` is the first setting to check for this project.

Fixes for common failures

Every endpoint returns HTTP 500

Set `Auth__CodeHmacKey` in the runtime environment used by the API, with at least 32 characters, then restart or recreate the app. On MonsterASP, add it in the website's Environment variables page. On Ubuntu, pass it to the container using the Compose override in step 4, or the GitHub secret and workflow change in step 6.

The project's `AuthService` throws when the key is missing or too short. Controllers that inject that service can fail before their endpoint action runs. This explains why a migration can succeed while API requests fail.

Only registration, resend-verification, or password-recovery email fails

Those endpoints also need working SMTP configuration. On MonsterASP, use the Gmail variables above or another external SMTP provider. For Ubuntu, use the email destination configured for that deployment; the local Compose development setup can route mail to Mailpit rather than a real inbox.

MonsterASP still returns 500 after adding the HMAC key

1. In the control panel, open **Websites > your website > Logs > Application Error** and read the exception at the time of the failed request.
2. If the normal log is not detailed enough, temporarily enable **ASP.NET Core debug** logging, reproduce the error, and read the stdout log. Disable debug logging after troubleshooting.
3. If the exception names a database login or connection error, compare `ConnectionStrings__DefaultConnection` with the connection string in the MonsterASP database page and confirm that it targets the same database you migrated.
4. Restart after each environment-variable change. MonsterASP applies them at application start.

MonsterASP's HTTP 500 guide and ASP.NET Core debug logging guide describe these log steps.

GitHub deployment stays queued or does not start

- **Listening for jobs:** normal idle status. A deployment starts only after a push to `master` or a manual workflow run.
- **Waiting for a runner:** make sure the Ubuntu VM is on, GitHub shows the runner Online, and its labels include `self-hosted`, `linux`, `x64`, and `hotel-vm`.
- **Runner is offline:** start the VM in VirtualBox, then run `cd ~/actions-runner && sudo ./svc.sh status && sudo ./svc.sh start` in Ubuntu.
- **The runner is Online but the job remains queued:** check the workflow repository. A runner registered at repository level must be registered under the same repository that owns the workflow.
- **"A session for this runner already exists":** stop a manually launched `./run.sh` with Ctrl+C, then restart the service with `sudo ./svc.sh stop` and `sudo ./svc.sh start`. Use the service or `./run.sh`, never both at once.
- **Changes are only on `Test`:** open a pull request with base `master` and compare `Test`, review it, then merge. The current workflow does not deploy from `Test` directly.

The Ubuntu API address or Termius connection changes

The Ubuntu VM can get a new LAN address after restart. Find it at the VM console with `ip -4 -br addr`, update the Termius host, and use that address in `http://<VM_LAN_IP>:8080/swagger`.